

DOM & Event Handling

JavaScript-ийн гол үндэс

Beginner – Intermediate түвшний хичээл

JavaScript



DOM гэж юу вэ? (Document Object Model)

DOM нь HTML документаг JavaScript-ээр удирдах боломж олгодог програмчлалын интерфейс юм. Браузер HTML-ийг уншаад модон бүтэц (tree) болгон хадгалдаг.

```
// JavaScript
```

```
// DOM-д хандах хамгийн энгийн жишээ  
console.log(document.title);  
// → "Миний Вэбсайт"
```

```
console.log(document.body);  
// → <body>...</body>
```

```
// DOM бол object (объект)  
console.log(typeof document);  
// → "object"
```

⚡ Хэрэглээ

Вэб хуудасны контент, загвар болон бүтцийг JavaScript-ээр динамикаар өөрчлөхөд ашигладаг.

👉 Анхаарах / Давуу тал

✓ Давуу: Хуудасны аль ч элементэд хандах боломжтой.

⚠ Анхаар: Их DOM manipulation → удаан гүйцэтгэл.

02 document ба window Object

window — браузерийн цонхыг илэрхийлнэ (глобал объект). document — HTML документаг илэрхийлнэ, window-ийн нэг хэсэг.

// JavaScript

```
// window → глобал орчин
console.log(window.innerWidth); // → 1440
console.log(window.location.href); // → поточ URL
```

```
// document → HTML документ
console.log(document.title);
console.log(document.URL);
```

```
// window.document === document
console.log(window.document === document);
// → true
```

```
// Alert, alert дуудах
window.alert("Сайн уу!");
alert("Ижил зүйл!"); // window. орхиж болно
```

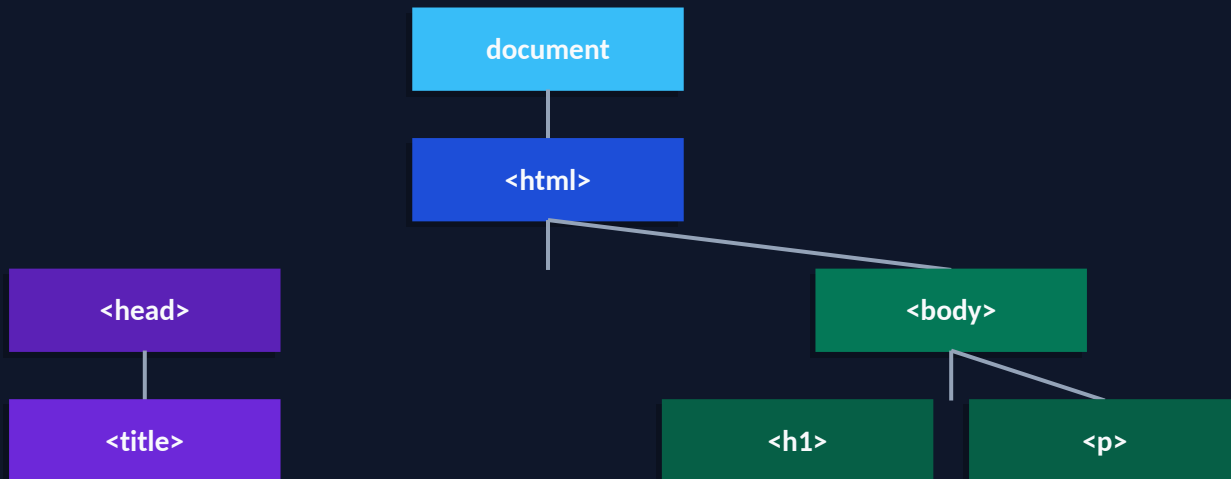
⚡ Хэрэглээ

window: цонхын хэмжээ, URL, navigation.
document: HTML элемент, контент.

👉 Анхаарах / Давуу тал

- ✓ document нь window-д байдаг.
- ⚠ Node.js-д window байхгүй, зөвхөн браузерт ажилладаг.

HTML файл уншигдахад браузер элемент бүрийг 'Node' болгон Tree бүтцээр зохион байгуулдаг.



Node төрлүүд:



Element Node (<tag>)



Text Node



Comment Node

querySelector() болон getElementById()

HTML элементийг JavaScript-ээр хайж олох хоёр үндсэн арга. getElementById нь хурдан, querySelector илүү уян хатан.

// JavaScript

```
// getElementById — ID-аар хайна
const title = document.getElementById('main-title');
console.log(title.textContent);

// querySelector — CSS selector ашиглана
const btn = document.querySelector('.btn-primary');
const first = document.querySelector('ul li:first-child');

// querySelectorAll — олон элемент буцаана
const items = document.querySelectorAll('.card');
items.forEach(item => {
  console.log(item.textContent);
});
```

```
// Олдоогүй бол → null буцаана
const el = document.querySelector('#ghost'); // null
```

⚡ Хэрэглээ

Товчлуур, форм, жагсаалт зэрэг элементэд хандахад тогтмол хэрэглэнэ.

🔗 Анхаарах / Давуу тал

- ✓ querySelector: CSS selector бүхний дэмжлэгтэй.
- ⚠ getElementById нь querySelector-оос арай хурдан.

addEventListener() болон removeEventListener()

Элемент дээр event listener нэмэх, устгах. Хэрэглэгчийн үйлдэлд (click, input, scroll...) хариу үзүүлэхэд хэрэглэнэ.

// JavaScript

```
const btn = document.querySelector('#myBtn');

// Event listener нэмэх
function handleClick(event) {
  console.log('Товчлуур дарагдлаа!', event);
}
btn.addEventListener('click', handleClick);

// Event listener устгах
btn.removeEventListener('click', handleClick);

// Нэргүй функц ашиглавал устгах боломжгүй!
btn.addEventListener('mouseover', (e) => {
  e.target.style.color = 'red'; // ✗ устгах боломжгүй
});
```

⚡ Хэрэглээ

Товч дарах, форм submit, гар дарах, скролл зэрэг бүх хэрэглэгчийн үйлдлийг удирдана.

📌 Анхаарах / Давуу тал

- ✓ removeEventListener-ийн тулд функцийг нэрлэж тодорхойл.
- ⚠ Memory leak-аас зайлсхийхийн тулд шаардлагагүй listener-ийг устга.

preventDefault()

Браузерийн анхдагч үйлдлийг зогсооно. Жишээ нь: форм submit хийх, холбоос дагах, баруун товчны цэс зэрэг.

// JavaScript

```
// Форм submit-ийг зогсоох
const form = document.querySelector('form');
form.addEventListener('submit', (event) => {
  event.preventDefault();
  // Өөрийн validation хийнэ
  console.log('Форм submit болсонгүй!');
});
```

```
// Холбоос дагахаас сэргийлэх
const link = document.querySelector('a');
link.addEventListener('click', (e) => {
  e.preventDefault();
  console.log('Шилжсэнгүй!');
});
```

```
// Баруун товчны цэсийг хаах
document.addEventListener('contextmenu', (e) => {
  e.preventDefault();
});
```

⚡ Хэрэглээ

Форм validation, custom navigation, drag-and-drop, keyboard shortcut тохируулахад.

👉 Анхаарах / Давуу тал

- ✓ Анхдагч үйлдлийг бүрэн хянах боломжтой.
- ⚠ Хэт ашиглавал хэрэглэгчийн туршлага муудна (жишээ: copy/paste хааж болохгүй).

stopPropagation()

Event-ийн дамжих (bubbling/capturing) явцыг зогсооно. Дотор элементийн event гадна элементэд хүрэхгүй болно.

// JavaScript

```
const outer = document.querySelector('.outer');  
const inner = document.querySelector('.inner');
```

```
outer.addEventListener('click', () => {  
  console.log('Гадна div дарагдлаа');  
});
```

```
inner.addEventListener('click', (e) => {  
  e.stopPropagation(); // bubbling зогсоо!  
  console.log('Дотор div дарагдлаа');  
  // → "Гадна div" гэж гарахгүй  
});
```

```
// stopPropagation байхгүй бол:  
// "Дотор div дарагдлаа"  
// "Гадна div дарагдлаа" ← ч гарна
```

⚡ Хэрэглээ

Nested element-д тус тусдаа click handler байх үед, modal, dropdown хаах механизм.

👉 Анхаарах / Давуу тал

- ✓ Event-ийн тархалтыг нарийн хянах боломжтой.
- ⚠ Хэт ашиглавал Event Delegation ажиллахгүй болж болно.

08 stopImmediatePropagation()

stopPropagation()-аас хүчтэй: мөн элемент дээрх бусад listener-уудыг ч зогсооно.

// JavaScript

```
const btn = document.querySelector("button");
```

```
btn.addEventListener('click', (e) => {  
  console.log('1-p handler');  
  e.stopImmediatePropagation();  
  // ← 2-p handler дуудагдахгүй  
});
```

```
btn.addEventListener('click', () => {  
  console.log('2-p handler'); // Ажиллахгүй!  
});
```

```
btn.addEventListener('click', () => {  
  console.log('3-p handler'); // Ажиллахгүй!  
});
```

// Зөвхөн "1-p handler" хэвлэгдэнэ

⚡ Хэрэглээ

Нэг элемент дээр олон listener байгаа үед, зөвхөн нэгийг нь ажиллуулах шаардлагатай үед.

🔗 Анхаарах / Давуу тал

✓ stopPropagation + дараагийн listener-ийг ч зогсооно.

⚠ Debugging хэцүү болгодог — зайлсхийж болох бол зайлс.

dispatchEvent()

JavaScript-ээс event-ийг программаар үүсгэж, элемент дээр гал асаана (fire/trigger хийнэ).

// JavaScript

```
const btn = document.querySelector('#myBtn');
```

```
// Listener нэмэх
```

```
btn.addEventListener('click', () => {  
  console.log('Товчлуур дарагдлаа!');  
});
```

```
// Гараар дарахгүйгээр event дуудах  
btn.dispatchEvent(new Event('click'));  
// → "Товчлуур дарагдлаа!"
```

```
// MouseEvent үүсгэх
```

```
const mouseEvent = new MouseEvent('click', {  
  bubbles: true,  
  cancelable: true  
});  
btn.dispatchEvent(mouseEvent);
```

⚡ Хэрэглээ

Unit test, автоматжуулалт, нэг event-ийг олон газраас дуудах хэрэгтэй үед.

👉 Анхаарах / Давуу тал

- ✓ Тест бичихэд маш хэрэгтэй.
- ⚠ dispatchEvent() синхрон ажилладаг (callback дуусч буцана).

Өөрийн event-ийг бүтээх боломж. detail property-ээр өгөгдөл дамжуулна. Component-үүдийн хоорондох харилцааг зохицуулна.

// JavaScript

```
// CustomEvent үүсгэх
const event = new CustomEvent('userLogin', {
  detail: {
    username: 'Батбаяр',
    role: 'admin'
  },
  bubbles: true,
  cancelable: true
});

// Listener нэмэх
document.addEventListener('userLogin', (e) => {
  console.log('Хэрэглэгч нэвтэрлээ:', e.detail.username);
  console.log('Эрх:', e.detail.role);
});

// Event дуудах
document.dispatchEvent(event);
```

⚡ Хэрэглээ

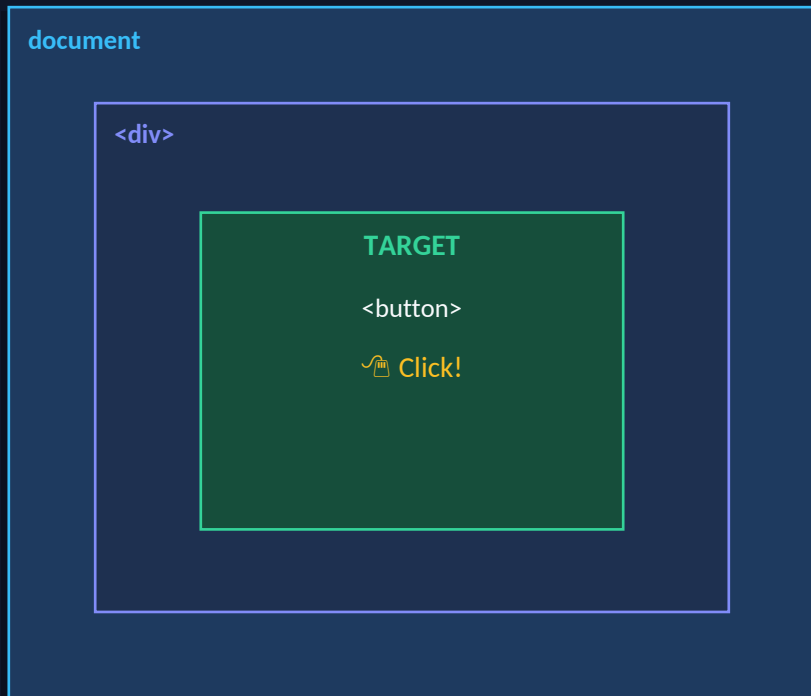
Component хоорондын мессеж дамжуулалт, custom notification систем, framework-гүй event bus.

👉 Анхаарах / Давуу тал

✓ Framework-гүйгээр component communication хийх хамгийн энгийн арга.
⚠ detail-д объект дамжуулахдаа reference-ийг анхаар.

Event Bubbling ба Event Capturing

Capturing: window→document дээрээс доош. Bubbling: target-аас дээш window хүртэл. addEventListener 3-р параметр: true=capture, false(default)=bubble.



CAPTURING BUBBLING
↓ доошоо ↑ дээшээ



// Bubbling vs Capturing

```
div.addEventListener(
  'click',
  handler,
  false // bubbling (default)
);
```

```
div.addEventListener(
  'click',
  handler,
  true // capturing
);
```

// Дарааллыг шалгах:
// 1. Capturing: top→down
// 2. Target
// 3. Bubbling: up→top

⚠ Ихэнх тохиолдолд Bubbling ашигладаг. Capturing нь ховор, тусгай тохиолдолд хэрэглэнэ.

Хүүхэд элемент бүрт listener нэмэхийн оронд эцэг элементэд нэг listener нэмэж, event-ийн bubbling-ийг ашиглана.

// JavaScript

```
// ✗ Муу арга: бүр item-д listener
document.querySelectorAll('.item').forEach(item => {
  item.addEventListener('click', handleClick);
}); // 100 item = 100 listener!
```

```
// ✔ Event Delegation: 1 listener
const list = document.querySelector('#todo-list');
list.addEventListener('click', (event) => {
  // Яг дарагдсан элементийг шалгана
  if (event.target.matches('.delete-btn')) {
    event.target.closest('li').remove();
  }
  if (event.target.matches('.edit-btn')) {
    // edit хийх
  }
});
// Динамикаар нэмэгдсэн item-д ч ажиллана!
```

⚡ Хэрэглээ

Todo list, хүснэгтийн мөрүүд, динамикаар нэмэгдэх элементүүд, их хэмжээний жагсаалт.

👉 Анхаарах / Давуу тал

✔ Memory хэмнэлттэй, динамик элементэд автоматаар ажилладаг.
⚠ event.target.matches() ашиглаж зөв элементийг шүүж авна.

JavaScript-ээр HTML элемент үүсгэх, устгах, өөрчлөх, загварчлах боломж. Динамик вэб хуудасны үндэс.

// JavaScript

```
// Элемент үүсгэх ба нэмэх
const li = document.createElement('li');
li.textContent = 'Шинэ item';
li.classList.add('active');
document.querySelector('ul').appendChild(li);
```

```
// innerHTML ашиглах
const div = document.querySelector('#app');
div.innerHTML = '<h2>Сайн уу!</h2>';
```

```
// Загвар өөрчлөх
const box = document.querySelector('.box');
box.style.backgroundColor = '#38BDF8';
box.style.display = 'none'; // нуух
```

```
// Устгах
const old = document.querySelector('.old-item');
old.remove();
```

⚡ Хэрэглээ

Динамик контент үүсгэх (чат, todo, хайлтын үр дүн), UI шинэчлэх.

🔗 Анхаарах / Давуу тал

- ✓ Аливаа UI-г JS-ээр бүтээх боломжтой.
- ⚠ innerHTML-д хэрэглэгчийн оролт шууд оруулбал XSS халдлага болно!

✓ ДАВУУ ТАЛ

- Хуудасны бүх элементэд хандах боломжтой
- Динамик контент үүсгэх, устгах, өөрчлөх боломжтой
- Event-driven програмчлал хийх боломжтой
- Браузер бүх дэмждэг — нэмэлт library хэрэггүй
- Форм validation, UI шинэчлэлт хялбархан
- CustomEvent-ээр component-үүдийг холбоно

⚠ СУЛ ТАЛ / АНХААРАХ

- ✗ Их DOM manipulation хуудасыг удаашруулдаг
- ✗ innerHTML-д XSS халдлагын эрсдэл байдаг
- ✗ Memory leak: listener-ийг устгахаа мартвал
- ✗ Reflow/Repaint: style өөрчлөлт үнэтэй байдаг
- ✗ Том апп-д DOM шууд удирдах нарийн болдог → Virtual DOM (React) гарч ирсэн



DOM = Tree бүтэц

HTML-ийг браузер object tree болгон хадгалдаг



Selector апра

querySelector, getElementById ашиглан элементэд хандана



Event Listener

addEventListener/removeEventListener — хэрэглэгчийн үйлдлийг удирдана



Event Control

preventDefault, stopPropagation, stopImmediatePropagation



Custom Events

dispatchEvent + CustomEvent — component хоорондын харилцаа

Bubbling & Capturing

Event дамжих хоёр чиглэл — Delegation-д ашиглана



Event Delegation

Нэг listener-ийн тусламжтай олон элементийг удирдана



DOM Manipulation

createElement, appendChild, remove, innerHTML

Асуулт?

Questions?

DOM & Event Handling
JavaScript

Баярлалаа! 🎉